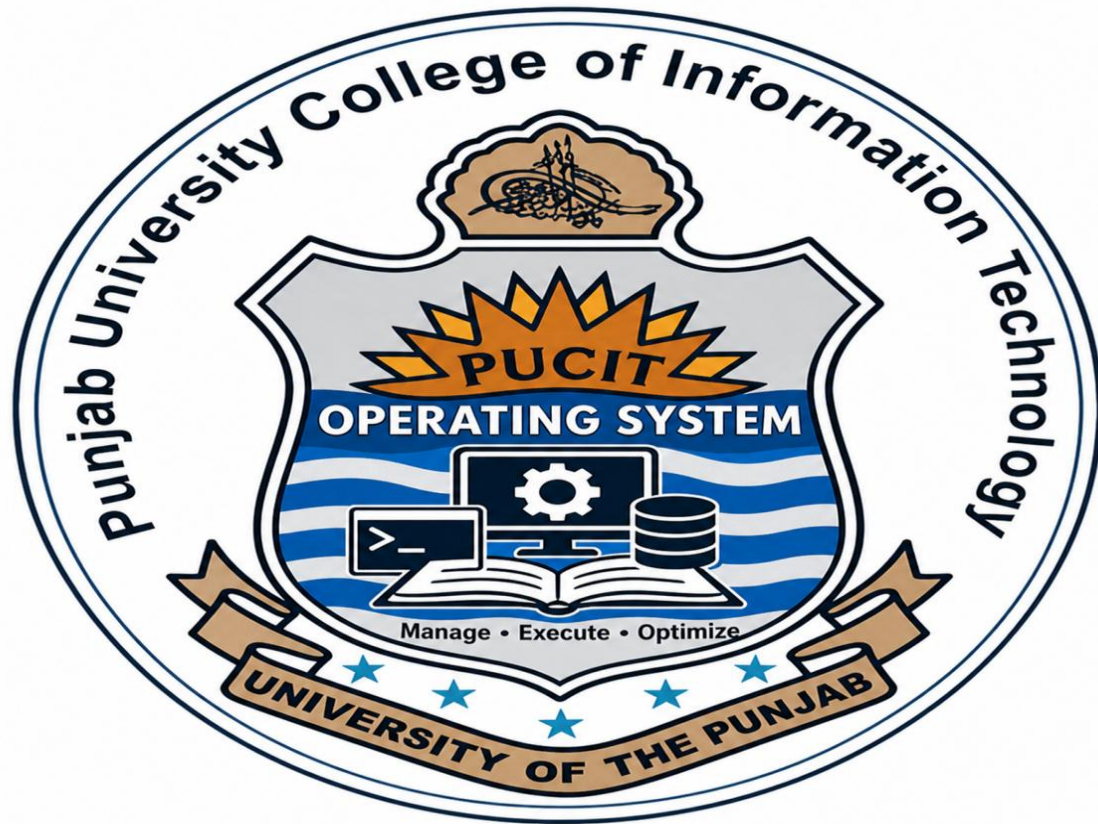




Name: Muhammad Abdul Kareem

Roll No: BSEF23M014

Section: BS-SE(Morning)

Semester: 6th

Assignment: 1st

Subject: Operating System

Submission Date: 05/04/2026

Submitted to:

Question 1

The following processes are being scheduled by the system. In addition to the processes listed below, the system also has an idle task (which consumes no CPU resources and is identified as idle). This task has priority 0 and is scheduled whenever the system has no other available processes to run.

- A. A pre-emptive round robin scheduling algorithm (ignore process priorities for this part). The length of a time quantum is 10 units.
- B. A pre-emptive priority scheduling algorithm. Each process is assigned a numerical priority, with a higher number indicating a higher priority.

Thread	Priority	Burst	Arrival
P1	40	20	0
P2	30	25	25
P3	30	25	30
P4	35	15	60
P5	5	10	100
P6	10	10	105

For both parts (part A and part B), do the following tasks:

- 1) Show the scheduling order of the processes using a Gantt chart.
- 2) What is the turnaround time for each process?
- 3) What is the waiting time for each process?
- 4) What is the CPU utilization rate?
- 5) Compare both scheduling algorithms in terms of average turn-around time.
- 6) Compare both scheduling algorithms in terms of average waiting time.

Solution 1 Part (A)

Task 1) Gantt Chart

Note:

When a process finishes its time quantum, newly arriving processes at that exact time are typically added to the ready queue before the preempted process is re-added.

P1 P1 Idle P2 P3 P2 P3 P2 P4 P3 P4 Idle P5 P6
0 10 20 25 35 45 55 65 70 80 85 90 100 110 120

Detailed:

- 0–10: P1 (remaining 10)
- 10–20: P1 (completes)
- 20–25: Idle (no ready process)
- 25–35: P2 (remaining 15)
- 35–45: P3 (remaining 15)
- 45–55: P2 (remaining 5)
- 55–65: P3 (remaining 5)
- 65–70: P2 (completes)
- 70–80: P4 (remaining 5)
- 80–85: P3 (completes)
- 85–90: P4 (completes)
- 90–100: Idle
- 100–110: P5 (completes)
- 110–120: P6 (completes)

Task 2) Turnaround Time (= Completion – Arrival)

<u>Process</u>	<u>Arrival</u>	<u>Completion</u>	<u>Turnaround</u>
P1	0	20	20
P2	25	70	45
P3	30	85	55
P4	60	90	30
P5	100	110	10
P6	105	120	15

Task 3) Waiting Time (= Turnaround – Burst)

<u>Process</u>	<u>Burst</u>	<u>Turnaround</u>	<u>Waiting</u>
P1	20	20	0
P2	25	45	20
P3	25	55	30
P4	15	30	15
P5	10	10	0
P6	10	15	5

Task 4) CPU Utilization

- ✚ **Total Time:** 120 units.
- ✚ **Idle Time:** 5 units (20-25) + 10 units (90-100) = 15 units.
- ✚ **Active Time:** 120 - 15 = 105 units.
- ✚ **Utilization:** $(105/120) \times 100 = 87.5\%$

Task 5) Average Turnaround

✚ $(20 + 45 + 55 + 30 + 10 + 15) / 6 = 175 / 6 \approx 29.17$

Comparison of both scheduling algorithms in terms of average turn-around time is at the end of **Solution 1 Part (B)**.

Task 6) Average Waiting

✚ $(0 + 20 + 30 + 15 + 0 + 5) / 6 = 70 / 6 \approx 11.67$

Comparison of both scheduling algorithms in terms of average turn-around time is at the end of **Solution 1 Part (B)**.

Solution 1 Part (B)

Task 1) Gantt Chart

P1	Idle	P2	P3	P4	P3	Idle	P5	P6	P5	
0	20	25	50	60	75	90	100	105	115	120

Detailed:

- 0–20: P1 (priority 40)
- 20–25: Idle
- 25–50: P2 (priority 30, completes; P3 arrives at 30 with same priority, waits)
- 50–60: P3 (priority 30)
- 60–75: P4 (priority 35, preempts P3)
- 75–90: P3 resumes and completes
- 90–100: Idle
- 100–105: P5 (priority 5)
- 105–115: P6 (priority 10, preempts P5 at 105)
- 115–120: P5 resumes and completes

Task 2) Turnaround Time (= Completion – Arrival)

<u>Process</u>	<u>Arrival</u>	<u>Completion</u>	<u>Turnaround</u>
P1	0	20	20
P2	25	50	25
P3	30	90	60
P4	60	75	15
P5	100	120	20
P6	105	115	10

Task 3) Waiting Time (= Turnaround – Burst)

<u>Process</u>	<u>Burst</u>	<u>Turnaround</u>	<u>Waiting</u>
P1	20	20	0
P2	25	25	0
P3	25	60	35
P4	15	15	0
P5	10	20	10
P6	10	10	0

Task 4) CPU Utilization

✚ **Total Time:** 120 units.

✚ **Idle Time:** 5 units (20-25) + 10 units (90-100) = 15 units.

✚ **Active Time:** 120 - 15 = 105 units.

✚ **Utilization:** $(105/120) \times 100 = 87.5\%$

Task 5) Average Turnaround

✚ $(20 + 25 + 60 + 15 + 20 + 10) / 6 = 150 / 6 \approx 25.00$

Comparison of both scheduling algorithms in terms of average turn-around time is at the end of this solution.

Task 6) Average Waiting

✚ $(0 + 0 + 35 + 0 + 10 + 0) / 6 = 45 / 6 = 7.50$

Comparison of both scheduling algorithms in terms of average turn-around time is at the end of *this solution*.

Comparison (A vs B)

<u>Metric</u>	<u>Round Robin (A)</u>	<u>Priority (B)</u>
Avg Turnaround	29.17	25.00
Avg Waiting	11.67	7.50

Average Turnaround Time:

Priority Scheduling (25.00) performs better than Round Robin (29.17). By favoring shorter/high-priority processes immediately rather than time-slicing, it completes processes faster on average.

Average Waiting Time:

Priority Scheduling (7.50) significantly outperforms Round Robin (11.67). Round Robin forces processes to continuously stop and wait in the queue, increasing overall wait time, whereas Priority allows most processes (except P3 and P5) to execute without waiting once they secure the CPU.

Conclusion:

Priority scheduling gives better average turnaround and waiting times for this specific workload.

Question 2

Write C/C++ code to implement round robin scheduling algorithm. Design the format of your input text file and explain in readme file. Process ID, arrival time, burst time and quantum should be input through text file. Each process's waiting time, response time and turn-around time should be displayed on screen and recorded to an output file too. Gantt chart should be displayed on screen only. Algorithm statistics like average response time, average turn-around time and average waiting time should also be displayed on screen.

Solution 2

Input File Design

Input File Format/Design:

The program uses a keyword-based format:

QUANTUM <time_quantum>

PROCESS <PID> <ArrivalTime> <BurstTime>

Example:

QUANTUM 4

PROCESS 1 0 10

PROCESS 2 1 4

PROCESS 3 2 6

PROCESS 4 4 3

PROCESS 5 6 8

PROCESS 6 8 2

Explanation:

- *QUANTUM defines time slice*
- *PROCESS defines each process*
- *Comments using # are ignored*

Algorithm Explanation

Algorithm: Round Robin Scheduling

1. Sort processes by arrival time
2. Use a queue to manage ready processes
3. Execute each process for a fixed quantum
4. If process is not finished, move it to the back of the queue
5. Record:
 - Response Time (first execution)
 - Completion Time
 - Turnaround Time
 - Waiting Time
6. Repeat until all processes complete

Source Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <queue>
#include <string>
#include <iomanip>
#include <algorithm>
```

```

#include <sstream>
#include <cstdlib>
#include <climits>

using namespace std;

// Data Structures
struct Process
{
    int pid;
    int arrival_time;
    int burst_time;
    // Computed fields
    int remaining_time;
    int waiting_time = 0;
    int turnaround_time = 0;
    int response_time = -1;    // -1 = not yet started
    int completion_time = 0;
    bool started = false;
};

struct GanttSlot
{
    int pid;           // -1 = CPU idle
    int start;
    int end;
};

// Read input file
bool readInput(const string& filename, vector<Process>& processes, int&
quantum)
{
    ifstream fin(filename);
    if (!fin.is_open())
    {
        cerr << "ERROR: Cannot open input file \"" << filename <<
"\n\n";
        return false;
    }
    string line;
    bool quantum_read = false;
    while (getline(fin, line))
    {
        // Strip inline comments (everything after #)
        size_t hash = line.find('#');
        if (hash != string::npos)
        {
            line = line.substr(0, hash);
        }
    }
}

```

```

// Trim leading/trailing whitespace
size_t first = line.find_first_not_of(" \t\r\n");
if (first == string::npos)
{
    continue;
}
line = line.substr(first);
istringstream iss(line);
string token;
iss >> token;
if (token == "QUANTUM")
{
    iss >> quantum;
    quantum_read = true;
}
else if (token == "PROCESS")
{
    Process p;
    iss >> p.pid >> p.arrival_time >> p.burst_time;
    p.remaining_time = p.burst_time;
    processes.push_back(p);
}
}
fin.close();
if (!quantum_read)
{
    cerr << "ERROR: QUANTUM not found in input file.\n";
    return false;
}
if (processes.empty())
{
    cerr << "ERROR: No PROCESS entries found in input file.\n";
    return false;
}
return true;
}

// Round Robin Scheduler
void runRoundRobin(vector<Process>& procs, int
quantum, vector<GanttSlot>& gantt)
{
    // Sort by arrival time (stable for equal arrivals → preserve file
order)
    sort(procs.begin(), procs.end(), [](const Process& a, const Process&
b)
    {
        return a.arrival_time < b.arrival_time;
    });
    int n = (int)procs.size();

```

```

int current_time = 0;
int completed = 0;
queue<int> ready;          // indices into procs[]
vector<bool> in_queue(n, false);
// Enqueue all processes that arrive at time 0
for (int i = 0; i < n; i++)
{
    if (procs[i].arrival_time == 0)
    {
        ready.push(i);
        in_queue[i] = true;
    }
}
while (completed < n)
{
    if (ready.empty())
    {
        // CPU idle: advance to next arrival
        int next_arr = INT32_MAX;
        for (int i = 0; i < n; i++)
        {
            if (!in_queue[i] && procs[i].remaining_time > 0)
            {
                next_arr = min(next_arr, procs[i].arrival_time);
            }
        }
        gantt.push_back({ -1, current_time, next_arr });
        current_time = next_arr;
        for (int i = 0; i < n; i++)
        {
            if (!in_queue[i] && procs[i].remaining_time > 0 &&
procs[i].arrival_time <= current_time)
            {
                ready.push(i);
                in_queue[i] = true;
            }
        }
        continue;
    }
    int idx = ready.front();
    ready.pop();
    // Record response time on first execution
    if (!procs[idx].started)
    {
        procs[idx].response_time = current_time -
procs[idx].arrival_time;
        procs[idx].started = true;
    }
    int exec_time = min(quantum, procs[idx].remaining_time);

```

```

        int slice_end = current_time + exec_time;
        gantt.push_back({ procs[idx].pid, current_time, slice_end });
        procs[idx].remaining_time -= exec_time;
        current_time = slice_end;
        // Enqueue newly arrived processes BEFORE re-adding current if
not done
        for (int i = 0; i < n; i++)
        {
            if (!in_queue[i] && procs[i].remaining_time > 0 &&
procs[i].arrival_time <= current_time)
            {
                ready.push(i);
                in_queue[i] = true;
            }
        }
        if (procs[idx].remaining_time > 0)
        {
            // Not finished: push back
            ready.push(idx);
        }
        else
        {
            // Finished
            procs[idx].completion_time = current_time;
            procs[idx].turnaround_time =
                procs[idx].completion_time - procs[idx].arrival_time;
            procs[idx].waiting_time =
                procs[idx].turnaround_time - procs[idx].burst_time;
            completed++;
        }
    }
}

// Display Gantt Chart (screen only)
void displayGantt(const vector<GanttSlot>& gantt)
{
    cout <<
"\n+=====+\n";
    cout << "|                                GANTT  CHART
|\n";
    cout <<
"+=====+\n\n";
    // Top bar
    cout << " ";
    for (const auto& slot : gantt)
    {
        int width = slot.end - slot.start;
        string label = (slot.pid == -1) ? "IDLE" : "P" +
to_string(slot.pid);

```

```

        int pad_total = max((int)label.size(), width * 2);
        cout << "+" << string(pad_total, '-') << "+";
    }
    cout << "\n ";
    for (const auto& slot : gantt)
    {
        int width = slot.end - slot.start;
        string label = (slot.pid == -1) ? "IDLE" : "P" +
to_string(slot.pid);
        int pad_total = max((int)label.size(), width * 2);
        int left_pad = (pad_total - (int)label.size()) / 2;
        int right_pad = pad_total - (int)label.size() - left_pad;
        cout << "|" << string(left_pad, ' ') << label
            << string(right_pad, ' ') << "|";
    }
    cout << "\n ";
    for (const auto& slot : gantt)
    {
        int width = slot.end - slot.start;
        string label = (slot.pid == -1) ? "IDLE" : "P" +
to_string(slot.pid);
        int pad_total = max((int)label.size(), width * 2);
        cout << "+" << string(pad_total, '-') << "+";
    }
    cout << "\n";
    // Time markers
    cout << " ";
    for (const auto& slot : gantt)
    {
        int width = slot.end - slot.start;
        string label = (slot.pid == -1) ? "IDLE" : "P" +
to_string(slot.pid);
        int pad_total = max((int)label.size(), width * 2);
        string t = to_string(slot.start);
        cout << t << string(pad_total - (int)t.size() + 1, ' ');
    }
    // Print last time
    cout << to_string(gantt.back().end) << "\n\n";
}

// Results Table (screen + file)
void displayResults(const vector<Process>& procs, int quantum, ostream&
out, bool is_screen)
{
    // Header
    out << "\n";
    if (is_screen)
    {

```

```

        out <<
"=====+\\n";
        out << " |          ROUND ROBIN SCHEDULING   (Quantum = " <<
quantum
        << " )          |\\n";
        out <<
"=====+\\n";
        out << " |  PID | Arrival | Burst | Completion | Turnaround |
Waiting | Response   |\\n";
        out <<
"=====+\\n";
    }
    else
    {
        out << "ROUND ROBIN SCHEDULING   (Quantum = " << quantum <<
")\\n";
        out << string(78, '=') << "\\n";
        out << left
            << setw(6) << "PID"
            << setw(10) << "Arrival"
            << setw(8) << "Burst"
            << setw(13) << "Completion"
            << setw(13) << "Turnaround"
            << setw(10) << "Waiting"
            << setw(10) << "Response"
            << "\\n";
        out << string(78, '-') << "\\n";
    }
    double sum_wt = 0, sum_tat = 0, sum_rt = 0;
    for (const auto& p : procs)
    {
        sum_wt += p.waiting_time;
        sum_tat += p.turnaround_time;
        sum_rt += p.response_time;
        if (is_screen)
        {
            out << " |  "
                << setw(3) << left << p.pid << " | "
                << setw(7) << right << p.arrival_time << " | "
                << setw(5) << right << p.burst_time << " | "
                << setw(10) << right << p.completion_time << " | "
                << setw(10) << right << p.turnaround_time << " | "
                << setw(7) << right << p.waiting_time << " | "
                << setw(8) << right << p.response_time << "   |\\n";
        }
    }
    else

```

```

        {
            out << left
                << setw(6) << p.pid
                << setw(10) << p.arrival_time
                << setw(8) << p.burst_time
                << setw(13) << p.completion_time
                << setw(13) << p.turnaround_time
                << setw(10) << p.waiting_time
                << setw(10) << p.response_time
                << "\n";
        }
    }
    int n = (int)procs.size();
    if (is_screen)
    {
        out <<
            "+=====+
            =====+\n";
        out << fixed << setprecision(2);
        out << "| Avg Waiting Time      : " << setw(8) << sum_wt / n <<
            " | \n";
        out << "| Avg Turnaround Time : " << setw(8) << sum_tat / n <<
            " | \n";
        out << "| Avg Response Time   : " << setw(8) << sum_rt / n <<
            " | \n";
        out <<
            "+=====+
            =====+\n";
    }
    else
    {
        out << string(78, '=') << "\n";
        out << fixed << setprecision(2);
        out << "Average Waiting Time      : " << sum_wt / n << "\n";
        out << "Average Turnaround Time : " << sum_tat / n << "\n";
        out << "Average Response Time   : " << sum_rt / n << "\n";
    }
}

// Main
int main(int argc, char* argv[])
{
    string input_file = "input.txt";
    string output_file = "output.txt";

    if (argc >= 2)
    {
        input_file = argv[1];
    }
}

```

```

    if (argc >= 3)
    {
        output_file = argv[2];
    }
    vector<Process> processes;
    int quantum = 0;

    cout << "Reading input from: " << input_file << "\n";
    if (!readInput(input_file, processes, quantum))
    {
        return 1;
    }
    cout << "Quantum = " << quantum << " | Processes loaded = " <<
processes.size() << "\n";

    vector<GanttSlot> gantt;
    runRoundRobin(processes, quantum, gantt);

    // Sort output by PID for readability
    sort(processes.begin(), processes.end(), [](const Process& a, const
Process& b) { return a.pid < b.pid; });

    // Screen output
    displayGantt(gantt);
    displayResults(processes, quantum, cout, true);

    // File output
    ofstream fout(output_file);
    if (!fout.is_open())
    {
        cerr << "WARNING: Cannot open output file \"" << output_file <<
"\n\n";
    }
    else
    {
        displayResults(processes, quantum, fout, false);
        fout.close();
        cout << "\nResults saved to: " << output_file << "\n";
    }
    return 0;
}

```

Input File

```
input.txt
File Edit View H1 ≡ B I S ↶ ⌂
#
# Round Robin Scheduler - Input File
# Format:
#   QUANTUM <time_quantum>
#   PROCESS <PID> <ArrivalTime> <BurstTime>
#
# Rules:
#   • Lines starting with # are comments and are ignored.
#   • Inline comments (after a value) are also supported.
#   • Fields are whitespace-separated.
#   • PID must be a unique positive integer.
#   • ArrivalTime >= 0, BurstTime > 0 (all integers).
#   • QUANTUM must appear before or after PROCESS lines – order does not matter.
#

QUANTUM 4          # Time quantum (context switch interval)

# PID  ArrivalTime  BurstTime
PROCESS 1      0      10
PROCESS 2      1       4
PROCESS 3      2       6
PROCESS 4      4       3
PROCESS 5      6       8
PROCESS 6      8       2
```

Output (SCREEN RESULT)

```
Microsoft Visual Studio Debu X + -
+=====+
+-----+
| P1 || P2 || P3 || P4 || P1 || P5 || P6 || P3 || P1 || P5 |
+-----+
| 0   4   8  12  15  19  23  25  27  29  33 |
+-----+

+=====+
|          ROUND ROBIN SCHEDULING (Quantum = 4)          |
+=====+
| PID | Arrival | Burst | Completion | Turnaround | Waiting | Response |
+=====+
| 1 | 0 | 10 | 29 | 29 | 19 | 0 |
| 2 | 1 | 4 | 8 | 7 | 3 | 3 |
| 3 | 2 | 6 | 27 | 25 | 19 | 6 |
| 4 | 4 | 3 | 15 | 11 | 8 | 8 |
| 5 | 6 | 8 | 33 | 27 | 19 | 13 |
| 6 | 8 | 2 | 25 | 17 | 15 | 15 |
+=====+
| Avg Waiting Time : 13.83 |
| Avg Turnaround Time : 19.33 |
| Avg Response Time : 7.50 |
+=====+

Results saved to: output.txt

C:\Users\muthe\OneDrive\Desktop\Projects\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 19824) exited with code 0 (0x 0).
Press any key to close this window . . .|
```

Output file

The output file (output.txt) contains the same table of results without the Gantt chart. This ensures that results are stored for record and submission.

```
input.txt  output.txt  ×  +
File Edit View H1 ⌵ ⋮ ⌵ B I

ROUND ROBIN SCHEDULING (Quantum = 4)
=====
PID  Arrival  Burst  Completion  Turnaround  Waiting  Response
-----
1    0        10    29          29          19        0
2    1         4     8           7           3         3
3    2         6    27          25          19        6
4    4         3    15          11           8         8
5    6         8    33          27          19       13
6    8         2    25          17          15       15
=====
Average Waiting Time : 13.83
Average Turnaround Time : 19.33
Average Response Time : 7.50
```

Readme File

➤ Overview

This program simulates the “Round Robin (RR)” CPU scheduling algorithm, one of the most widely used preemptive scheduling policies in time-sharing operating systems. Each process is given a fixed time slice called the “time quantum (Q)”. When a process exhausts its quantum without finishing, it is preempted and moved to the back of the ready queue so the next waiting process can run.

➤ Input File Format

The input file uses a “keyword-based plain-text format” that is both human-readable and easy to parse.

➤ Syntax Rules

QUANTUM <time_quantum>
PROCESS <PID> <ArrivalTime> <BurstTime>

<u>Keyword</u>	<u>Field</u>	<u>Type</u>	<u>Constraint</u>
Quantum	Time_quantum	Integer	> 0
Process	PID	Integer	Unique, > 0
Process	Arrivaltime	Integer	>= 0
Process	Bursttime	Integer	> 0

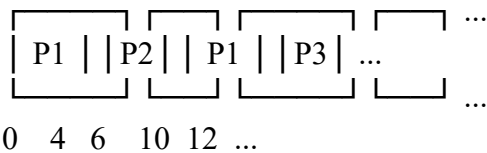
➤ Design Rationale

A keyword-based format was chosen over a raw CSV or fixed-column format for the following reasons:

1. **Self-documenting**:- 'PROCESS' and 'QUANTUM' make each line's purpose immediately obvious without a header.
2. **Order-independent**:- The 'QUANTUM' line may appear anywhere in the file.
3. **Extensible**:- New keywords (e.g., 'PRIORITY') can be added in future without breaking existing parsers.
4. **Comment support**:- Instructors and students can annotate the file freely without affecting parsing.

➤ Output Description

1. **Gantt Chart**:- Visual timeline showing which process ran during each time slice. Idle CPU periods are labeled 'IDLE'.



2. **Per-process table**:- showing:
 - PID, Arrival Time, Burst Time
 - Completion Time
 - **Turnaround Time** = Completion – Arrival
 - **Waiting Time** = Turnaround – Burst
 - **Response Time** = First CPU Start – Arrival
3. **Algorithm Statistics**:-
 - Average Waiting Time
 - Average Turnaround Time
 - Average Response Time

➤ Formulas

<u>Metric</u>	<u>Formula</u>
Turnaround Time	Completion Time – Arrival Time
Waiting Time	Turnaround Time – Burst Time
Response Time	First CPU Execution Start – Arrival
Avg Waiting Time	$\Sigma \text{Waiting Time} / n$
Avg Turnaround	$\Sigma \text{Turnaround Time} / n$
Avg Response Time	$\Sigma \text{Response Time} / n$

Question 3

Write (C/C++ or Python or Java) code to implement a multi-level feedback queue (MLFQ) scheduling algorithm with given specifications.

Queue No.	Quantum	Algorithm	Queue Priority
Q1	5 msec	RR	1
Q2	8 msec	RR	2
Q3	-	SRTF	3

- 1) Input details of n number of processes from a text file. Process details will include process ID, arrival time, burst time and priority value. Use appropriate data structures to save process details after reading from file. Then use appropriate data structures to implement three ready queues and generate time line (Gantt Chart).

Solution 3

Queue Configuration

The configuration of MultiLevel Queues according to the above table are as follows:

1. Q1 → Round Robin (Quantum = 5 ms) → Highest Priority
2. Q2 → Round Robin (Quantum = 8 ms) → Medium Priority
3. Q3 → Shortest Remaining Time First (SRTF) → Lowest Priority

Input File Design

Input File Format:

PROCESS <PID> <ArrivalTime> <BurstTime> <Priority>

Explanation:

- PID: Process ID
- ArrivalTime: Time when process arrives
- BurstTime: CPU execution time
- Priority:
 - 1 → Q1
 - 2 → Q2
 - 3 → Q3

Example:

```
PROCESS 1 0 12 1
PROCESS 2 2 5 1
PROCESS 3 4 15 2
PROCESS 4 6 3 1
PROCESS 5 8 20 3
PROCESS 6 10 7 2
PROCESS 7 12 4 1
```

Algorithm Explanation

Algorithm: Multi-Level Feedback Queue (MLFQ)

1. Processes are assigned to queues based on priority.
2. Q1 has highest priority, followed by Q2 and Q3.
3. Scheduling rules:
 - Q1 → Round Robin (5 ms)
 - Q2 → Round Robin (8 ms)
 - Q3 → SRTF
4. Preemption:
 - Higher-priority queues interrupt lower ones.
 - Q3 uses shortest remaining time for selection.
5. Demotion:
 - If a process exhausts its quantum:
 - Q1 → Q2
 - Q2 → Q3
6. Execution continues until all processes finish.

Data Structures Used

- vector → store all processes
- deque → implement Q1, Q2, Q3
- GanttSlot → store execution timeline

Source Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <deque>
#include <algorithm>
#include <iomanip>
#include <string>
#include <sstream>
#include <climits>

using namespace std;

// Data Structures
struct Process
{
    int pid;
    int arrival_time;
    int burst_time;
    int priority;           // initial queue: 1, 2, or 3
};
```

```

// Runtime tracking
int remaining_time;
int current_queue; // queue it currently sits in
int quantum_used = 0; // time used in current quantum slice
// Statistics (computed)
int waiting_time = 0;
int turnaround_time = 0;
int response_time = -1; // -1 = never scheduled yet
int completion_time = 0;
bool finished = false;
};

struct GanttSlot
{
    int pid; // -1 = CPU idle
    int queue; // 0 = idle
    int start;
    int end;
};

// Read Input File
bool readInput(const string& filename, vector<Process>& procs)
{
    ifstream fin(filename);
    if (!fin.is_open())
    {
        cerr << "ERROR: Cannot open \"" << filename << "\"\n";
        return false;
    }
    string line;
    while (getline(fin, line))
    {
        size_t h = line.find('#');
        if (h != string::npos)
        {
            line = line.substr(0, h);
        }
        size_t f = line.find_first_not_of(" \t\r\n");
        if (f == string::npos)
        {
            continue;
        }
        line = line.substr(f);
        istringstream iss(line);
        string token; iss >> token;
        if (token == "PROCESS")
        {
            Process p{};
            iss >> p.pid >> p.arrival_time >> p.burst_time >> p.priority;

```

```

        p.priority = max(1, min(3, p.priority)); // clamp 1-3
        p.remaining_time = p.burst_time;
        p.current_queue = p.priority;
        procs.push_back(p);
    }
}
fin.close();
if (procs.empty())
{
    cerr << "ERROR: No PROCESS entries found.\n";
    return false;
}
return true;
}

// Helper: find SRTF process index in Q3 list
int srtfIndex(const deque<int>& q3, const vector<Process>& procs)
{
    int best = -1;
    for (int idx : q3)
    {
        if (best == -1 || procs[idx].remaining_time <
            procs[best].remaining_time || (procs[idx].remaining_time ==
            procs[best].remaining_time && procs[idx].arrival_time <
            procs[best].arrival_time))
        {
            best = idx;
        }
    }
    return best;
}

// MLFQ Simulation
void runMLFQ(vector<Process>& procs, vector<GanttSlot>& gantt)
{
    const int Q1_QUANTUM = 5;
    const int Q2_QUANTUM = 8;

    int n = (int)procs.size();

    // Sort by arrival time for processing convenience
    sort(procs.begin(), procs.end(), [](const Process& a, const Process& b)
    {
        return a.arrival_time < b.arrival_time;
    });

    deque<int> q1, q2, q3; // indices into procs[]
    vector<bool> enqueued(n, false);

```

```

auto enqueueArrivals = [&](int t)
{
    for (int i = 0; i < n; i++)
    {
        if (!enqueued[i] && !procs[i].finished && procs[i].arrival_time
<= t)
        {
            enqueued[i] = true;
            // Push to its initial queue
            if (procs[i].current_queue == 1)
            {
                q1.push_back(i);
            }
            else if (procs[i].current_queue == 2)
            {
                q2.push_back(i);
            }
            else
            {
                q3.push_back(i);
            }
        }
    }
};

int current_time = 0;
int completed = 0;
int running = -1;    // index of currently running process, -1=idle

// Helper: add Gantt slot or extend last one
auto addGantt = [&](int pid, int queue, int t)
{
    if (!gantt.empty() && gantt.back().pid == pid && gantt.back().queue
== queue && gantt.back().end == t)
    {
        gantt.back().end = t + 1;
    }
    else
    {
        gantt.push_back({ pid, queue, t, t + 1 });
    }
};

// Find the maximum possible end time to bound simulation
int max_time = 0;
for (auto& p : procs)
{
    max_time += p.burst_time + p.arrival_time;
}
max_time += 10; // safety margin

```

```

while (completed < n && current_time < max_time)
{
    // 1. Enqueue newly arrived processes
    enqueueArrivals(current_time);
    // 2. Determine active process for this tick
    //     Priority: Q1 > Q2 > Q3
    //     SRTF for Q3 means we always re-pick shortest each tick.
    // Preemption check: if running Q2/Q3 and Q1 is now non-empty →
preempt
    if (running != -1)
    {
        int rq = procs[running].current_queue;
        if (rq == 2 && !q1.empty())
        {
            // Preempt: put back at front of Q2
            q2.push_front(running);
            procs[running].quantum_used = 0;
            running = -1;
        }
        else if (rq == 3 && (!q1.empty() || !q2.empty()))
        {
            // Preempt: put back into Q3 (SRTF will re-pick)
            q3.push_back(running);
            running = -1;
        }
    }
    // 3. Pick next process to run if CPU is free
    if (running == -1)
    {
        if (!q1.empty())
        {
            running = q1.front(); q1.pop_front();
            procs[running].quantum_used = 0;
        }
        else if (!q2.empty())
        {
            running = q2.front(); q2.pop_front();
            procs[running].quantum_used = 0;
        }
        else if (!q3.empty())
        {
            // SRTF: pick shortest remaining from Q3
            int best = srtfIndex(q3, procs);
            q3.erase(find(q3.begin(), q3.end(), best));
            running = best;
        }
    }
    // For SRTF in Q3: every tick re-evaluate if a shorter process exists

```

```

        if (running != -1 && procs[running].current_queue == 3 &&
!q3.empty())
        {
            int best = srtfIndex(q3, procs);
            if (procs[best].remaining_time < procs[running].remaining_time)
            {
                // Preempt current Q3 process
                q3.push_back(running);
                q3.erase(find(q3.begin(), q3.end(), best));
                running = best;
            }
        }
// 4. Execute 1 tick
if (running == -1)
{
    // CPU idle
    addGantt(-1, 0, current_time);
    current_time++;
    continue;
}
Process& rp = procs[running];
// Record response time (first time scheduled)
if (rp.response_time == -1)
{
    rp.response_time = current_time - rp.arrival_time;
}
addGantt(rp.pid, rp.current_queue, current_time);
rp.remaining_time--;
rp.quantum_used++;
current_time++;
// 5. Check if process finished
if (rp.remaining_time == 0)
{
    rp.finished = true;
    rp.completion_time = current_time;
    rp.turnaround_time = rp.completion_time - rp.arrival_time;
    rp.waiting_time = rp.turnaround_time - rp.burst_time;
    completed++;
    running = -1;
    continue;
}
// 6. Check quantum expiry (Q1 / Q2)
if (rp.current_queue == 1 && rp.quantum_used >= Q1_QUANTUM)
{
    rp.current_queue = 2;
    rp.quantum_used = 0;
    q2.push_back(running);
    running = -1;
}

```

```

        else if (rp.current_queue == 2 && rp.quantum_used >= Q2_QUANTUM)
        {
            rp.current_queue = 3;
            rp.quantum_used = 0;
            q3.push_back(running);
            running = -1;
        }
        // Q3 (SRTF) has no quantum – runs until preempted or done
    }
}

// Display Gantt Chart (screen only)
void displayGantt(const vector<GanttSlot>& gantt)
{
    cout <<
    "\n+=====+\n";
    cout << "|                MLFQ GANTT CHART    (Q1 | Q2 | Q3/SRTF)
    |\n";
    cout <<
    "+=====+\n\n";

    // Merge adjacent same-process slots for display
    vector<GanttSlot> disp;
    for (const auto& s : gantt)
    {
        if (!disp.empty() && disp.back().pid == s.pid && disp.back().queue ==
s.queue && disp.back().end == s.start)
        {
            disp.back().end = s.end;
        }
        else
        {
            disp.push_back(s);
        }
    }
    // Top border
    cout << " ";
    for (const auto& s : disp)
    {
        int dur = s.end - s.start;
        string lbl = (s.pid == -1) ? "IDLE" : "P" + to_string(s.pid);
        int w = max((int)lbl.size() + 2, dur * 2);
        cout << "+" << string(w, '-');
    }
    cout << "+\n ";
    // Labels row
    for (const auto& s : disp)
    {
        int dur = s.end - s.start;

```

```

        string lbl = (s.pid == -1) ? "IDLE" : "P" + to_string(s.pid);
        int w = max((int)lbl.size() + 2, dur * 2);
        int lp = (w - (int)lbl.size()) / 2;
        int rp = w - (int)lbl.size() - lp;
        cout << "|" << string(lp, ' ') << lbl << string(rp, ' ');
    }
    cout << "|\\n ";
    // Queue label row
    for (const auto& s : disp)
    {
        int dur = s.end - s.start;
        string lbl = (s.pid == -1) ? "IDLE" : "P" + to_string(s.pid);
        int w = max((int)lbl.size() + 2, dur * 2);
        string qlbl = (s.queue == 0) ? "" : "Q" + to_string(s.queue);
        int lp = (w - (int)qlbl.size()) / 2;
        int rp = w - (int)qlbl.size() - lp;
        cout << "|" << string(lp, ' ') << qlbl << string(rp, ' ');
    }
    cout << "|\\n ";
    // Bottom border
    for (const auto& s : disp)
    {
        int dur = s.end - s.start;
        string lbl = (s.pid == -1) ? "IDLE" : "P" + to_string(s.pid);
        int w = max((int)lbl.size() + 2, dur * 2);
        cout << "+" << string(w, '-');
    }
    cout << "+\\n";
    // Time markers
    cout << " ";
    for (size_t i = 0; i < disp.size(); i++)
    {
        int dur = disp[i].end - disp[i].start;
        string lbl = (disp[i].pid == -1) ? "IDLE" : "P" +
to_string(disp[i].pid);
        int w = max((int)lbl.size() + 2, dur * 2);
        string t = to_string(disp[i].start);
        cout << t << string(w - (int)t.size() + 1, ' ');
    }
    cout << to_string(disp.back().end) << "\\n\\n";
}

// Results Table (screen + file)
void displayResults(const vector<Process>& procs, ostream& out, bool screen)
{
    double sw = 0, stat = 0, srt = 0;
    int n = (int)procs.size();

    if (screen)

```

```

        {
            out <<
"\n+=====
==+\n";
            out << "|                      MLFQ SCHEDULING - PER-PROCESS STATISTICS
|\n";
            out <<
"+=====
+\n";
            out << "| PID | InitQ | Arrival | Burst | Completion | Turnaround |
Waiting | Resp |\n";
            out <<
"+=====
+\n";
        }
        else
        {
            out << "MLFQ SCHEDULING - PER-PROCESS STATISTICS\n";
            out << string(80, '=') << "\n";
            out << left
                << setw(5) << "PID"
                << setw(7) << "InitQ"
                << setw(9) << "Arrival"
                << setw(7) << "Burst"
                << setw(12) << "Completion"
                << setw(12) << "Turnaround"
                << setw(9) << "Waiting"
                << setw(9) << "Response"
                << "\n" << string(80, '-') << "\n";
        }
    for (const auto& p : procs)
    {
        sw += p.waiting_time;
        stat += p.turnaround_time;
        srt += p.response_time;
        if (screen)
        {
            out << "| " << setw(3) << left << p.pid
                << " |   Q" << p.priority
                << " | " << setw(7) << right << p.arrival_time
                << " | " << setw(5) << right << p.burst_time
                << " | " << setw(10) << right << p.completion_time
                << " | " << setw(10) << right << p.turnaround_time
                << " | " << setw(7) << right << p.waiting_time
                << " | " << setw(4) << right << p.response_time
                << " |\n";
        }
        else
        {

```

```

        out << left
            << setw(5) << p.pid
            << setw(7) << ("Q" + to_string(p.priority))
            << setw(9) << p.arrival_time
            << setw(7) << p.burst_time
            << setw(12) << p.completion_time
            << setw(12) << p.turnaround_time
            << setw(9) << p.waiting_time
            << setw(9) << p.response_time
            << "\n";
    }
}
if (screen)
{
    out <<
    "+=====
+\\n";

    out << fixed << setprecision(2);
    out << "| Avg Waiting Time      : " << setw(7) << sw / n << "
    |\\n";

    out << "| Avg Turnaround Time : " << setw(7) << stat / n << "
    |\\n";

    out << "| Avg Response Time   : " << setw(7) << srt / n << "
    |\\n";

    out <<
    "+=====
+\\n";
    }
    else
    {
        out << string(80, '=') << "\\n" << fixed << setprecision(2);
        out << "Average Waiting Time      : " << sw / n << "\\n";
        out << "Average Turnaround Time : " << stat / n << "\\n";
        out << "Average Response Time   : " << srt / n << "\\n";
    }
}

// Main
int main(int argc, char* argv[])
{
    string infile = "input.txt";
    string outfile = "output.txt";
    if (argc >= 2)
    {
        infile = argv[1];
    }
    if (argc >= 3)
    {
        outfile = argv[2];
    }
}

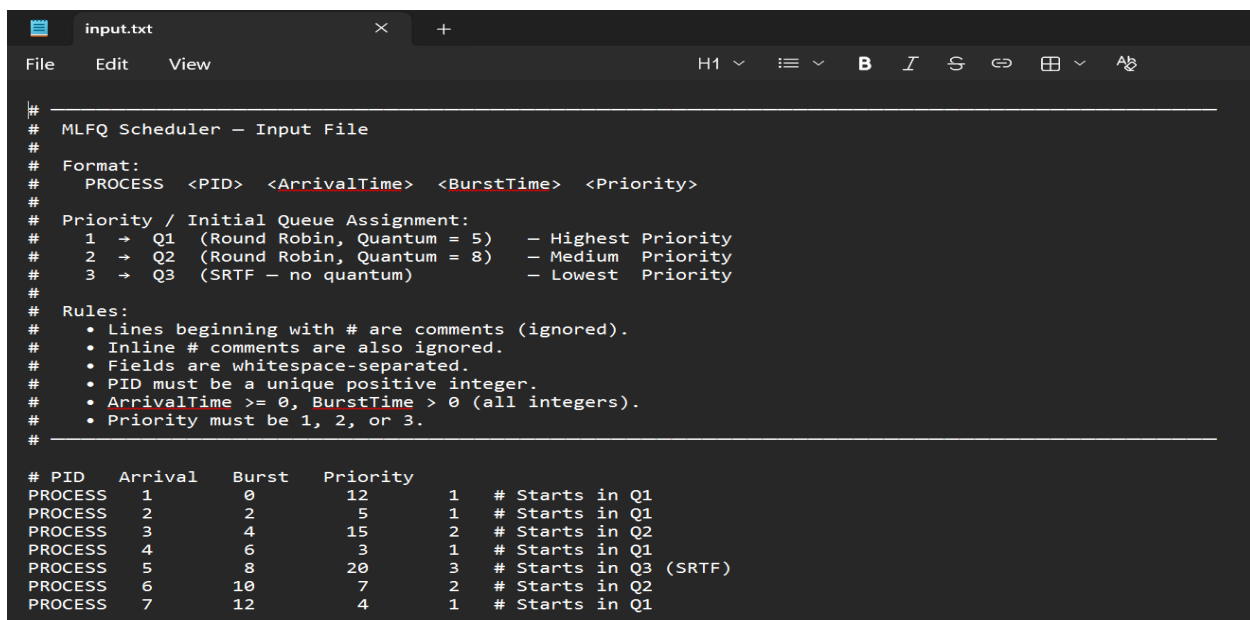
```

```

    }
    vector<Process> procs;
    cout << "Reading from: " << infile << "\n";
    if (!readInput(infile, procs))
    {
        return 1;
    }
    cout << "Processes loaded: " << procs.size() << "\n";
    vector<GanttSlot> gantt;
    runMLFQ(procs, gantt);
    // Sort by PID for display
    sort(procs.begin(), procs.end(), [](const Process& a, const Process& b)
        { return a.pid < b.pid; }
    );
    displayGantt(gantt);
    displayResults(procs, cout, true);
    ofstream fout(outfile);
    if (fout.is_open())
    {
        displayResults(procs, fout, false);
        fout.close();
        cout << "\nResults written to: " << outfile << "\n";
    }
    else
    {
        cerr << "WARNING: Cannot open output file.\n";
    }
    return 0;
}

```

Input File



```

#
# MLFQ Scheduler - Input File
#
# Format:
#   PROCESS <PID> <ArrivalTime> <BurstTime> <Priority>
#
# Priority / Initial Queue Assignment:
#   1 -> Q1 (Round Robin, Quantum = 5) - Highest Priority
#   2 -> Q2 (Round Robin, Quantum = 8) - Medium Priority
#   3 -> Q3 (SRTF - no quantum) - Lowest Priority
#
# Rules:
#   • Lines beginning with # are comments (ignored).
#   • Inline # comments are also ignored.
#   • Fields are whitespace-separated.
#   • PID must be a unique positive integer.
#   • ArrivalTime >= 0, BurstTime > 0 (all integers).
#   • Priority must be 1, 2, or 3.
#
# PID  Arrival  Burst  Priority
PROCESS 1      0      12      1 # Starts in Q1
PROCESS 2      2       5      1 # Starts in Q1
PROCESS 3      4     15      2 # Starts in Q2
PROCESS 4      6       3      1 # Starts in Q1
PROCESS 5      8     20      3 # Starts in Q3 (SRTF)
PROCESS 6     10       7      2 # Starts in Q2
PROCESS 7     12       4      1 # Starts in Q1

```

Output (SCREEN RESULT)/ Gantt Chart

```
Microsoft Visual Studio Debug Console
Reading from: input.txt
Processes loaded: 7

=====+
|          MLFQ GANTT CHART (Q1 | Q2 | Q3/SRTF)          |
+=====+

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| P1    | P2    | P4    | P7    | P3    | P1    | P6    | P3    | P5    |
| Q1    | Q1    | Q1    | Q1    | Q2    | Q2    | Q2    | Q3    | Q3    |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 0      | 5      | 10     | 13     | 17     | 25     | 32     | 39     | 46     | 66     |
+-----+-----+-----+-----+-----+-----+-----+-----+

=====+
|          MLFQ SCHEDULING & PER-PROCESS STATISTICS          |
+=====+
| PID | InitQ | Arrival | Burst | Completion | Turnaround | Waiting | Resp |
+=====+
| 1   | Q1    | 0       | 12    | 32          | 32          | 20      | 0     |
| 2   | Q1    | 2       | 5     | 10          | 8           | 3       | 3     |
| 3   | Q2    | 4       | 15    | 46          | 42          | 27      | 13    |
| 4   | Q1    | 6       | 3     | 13          | 7           | 4       | 4     |
| 5   | Q3    | 8       | 20    | 66          | 58          | 38      | 38    |
| 6   | Q2    | 10      | 7     | 39          | 29          | 22      | 22    |
| 7   | Q1    | 12      | 4     | 17          | 5           | 1       | 1     |
+=====+
| Avg Waiting Time   : 16.43 |
| Avg Turnaround Time : 25.86 |
| Avg Response Time  : 11.57 |
+=====+

Results written to: output.txt

C:\Users\muthe\OneDrive\Desktop\Projects\ConsoleApplication2\x64\Debug\ConsoleApplication2.exe (process 18604) exited with code 0 (0x0).
Press any key to close this window . . .
```

Output file

```
input.txt  output.txt
File Edit View H1 ≡ B I S

MLFQ SCHEDULING - PER-PROCESS STATISTICS
=====
PID  InitQ  Arrival  Burst  Completion  Turnaround  Waiting  Response
-----
1    Q1    0        12     32          32          20       0
2    Q1    2        5      10          8           3       3
3    Q2    4        15     46          42          27      13
4    Q1    6        3      13          7           4       4
5    Q3    8        20     66          58          38      38
6    Q2    10       7      39          29          22      22
7    Q1    12       4      17          5           1       1
=====
Average Waiting Time   : 16.43
Average Turnaround Time : 25.86
Average Response Time  : 11.57
|
```

Readme File

➤ Queue Configuration

<u>Queue</u>	<u>Algorithm</u>	<u>Quantum</u>	<u>Priority</u>
Q1	Round Robin	5 ms	1 (Highest)
Q2	Round Robin	8 ms	2 (Medium)
Q3	SRTF	---	3 (Lowest)

➤ Syntax Rules

PROCESS <PID> <ArrivalTime> <BurstTime> <Priority>

<u>Field</u>	<u>Type</u>	<u>Constraint</u>
Priority	Integer	1, 2, or 3
PID	Integer	Unique, > 0
Arrivaltime	Integer	≥ 0
Bursttime	Integer	> 0

➤ MLFQ Rules Implemented

1. **Three queues:** Q1 (RR/Q=5), Q2 (RR/Q=8), Q3 (SRTF).
2. **Initial queue:** determined by the 'priority' field in the input.
3. **Demotion only:** a process that exhausts its quantum in Q1 moves to Q2; from Q2 to Q3. Processes never move up.
4. **Preemption between queues:** a process running in Q2 or Q3 is immediately preempted when a process enters a higher-priority queue.
5. **SRTF within Q3:** every clock tick, the process with the smallest remaining burst time is selected. Ties broken by earliest arrival time.
6. **Quantum reset on preemption:** if a process is preempted mid-quantum by a higher-priority arrival, its quantum usage resets when it resumes.

➤ Loop Simulation

current_time = 0

while (completed < n):

1. Enqueue all processes that arrived by current_time into their initial queue.
2. Preemption check:
 - If running Q2 and Q1 non-empty → preempt, re-queue at front of Q2.
 - If running Q3 and Q1/Q2 non-empty → preempt, re-queue into Q3.
 - Within Q3: if a shorter process exists → SRTF preemption.
3. Pick next process (if CPU idle):
Q1 front → Q2 front → Q3 shortest remaining time

4. Execute 1 clock tick:
 - Record response time on first scheduling.
 - Decrement remaining_time, increment quantum_used.
5. If process finishes:
 - Record completion, turnaround, waiting times.
6. If quantum expires (Q1 or Q2):
 - Demote to next queue, reset quantum_used.

➤ Data Structure

<u>Structure</u>	<u>Purpose</u>
vector<Process>	All process records (read from input)
deque<int> q1	Q1 ready queue (FIFO for Round Robin)
deque<int> q2	Q2 ready queue (FIFO for Round Robin)
deque<int> q3	Q3 pool (SRTF — picked by shortest RT)
vector<GanttSlot>	Gantt timeline (pid, queue, start, end)

➤ Formulas

<u>Metric</u>	<u>Formula</u>
Turnaround Time	Completion Time – Arrival Time
Waiting Time	Turnaround Time – Burst Time
Response Time	First CPU Execution Start – Arrival
Avg Waiting Time	$\Sigma \text{Waiting Time} / n$
Avg Turnaround	$\Sigma \text{Turnaround Time} / n$
Avg Response Time	$\Sigma \text{Response Time} / n$

